

The story to tell on stage

Mary, 74, is being discharged after a hip operation.

- **Dr. Rao (senior)** writes the discharge summary: continue metformin 500 mg twice daily, continue sertraline 50 mg once daily, start Dolo 650 three times daily for pain.
- **Dr. Lee (junior)**, finishing a 12-hour shift with six other discharges, writes the prescription: metformin 1000 mg twice daily, paracetamol 650 mg three times daily. Sertraline doesn't make it onto the prescription at all.

Nobody notices. Mary would go home on double her diabetes dose, without her antidepressant, and with a generic substituted for the brand her doctor chose.

ScriptGuard catches all three within seconds of the files being saved. The pharmacist sees two high-severity flags and one low-severity "confirm brand" flag, calls Dr. Lee, and fixes the prescription before Mary leaves.

Why every mismatch is flagged, even brand vs generic

Doctors sometimes choose a specific brand on purpose and don't write down why. Real reasons:

- **Inactive ingredients differ between brands.** Fillers, dyes, lactose, or gluten vary by manufacturer. A patient sensitive to one may need a specific brand.
- **Some drugs are risky to switch between manufacturers.** For narrow-margin drugs such as levothyroxine (thyroid), tacrolimus (transplant), and some epilepsy medicines, small differences between versions can matter, so doctors may insist on one brand.
- **"Dispense as written."** Doctors can mark a prescription as no-substitution without giving a reason. Busy doctors rarely explain.

So ScriptGuard never silently treats two names as equivalent. It flags the difference and grades it: a

brand vs generic difference with the same ingredient and dose is low severity ("confirm brand intent"); anything else is high.

Why local and offline

- Patient documents never leave the building. No cloud contract or data-sharing negotiation needed.
- Works when the hospital network or internet is down.
- Predictable one-time hardware cost instead of per-document cloud fees.
- OpenShell's sandbox policy enforces it: the agent has no internet and can only touch one folder. That's provable, not just promised.

What NOT to say on stage

- **Don't say "Dolo is avoided because of a paracetamol allergy."** Dolo 650 *is* paracetamol. Its only active ingredient is paracetamol. Use the real reasons above.
- **Don't say ScriptGuard "blocks orders."** It flags them for a pharmacist. The human decides.
- **Don't say "CLEAN" means safe.** Say "no mismatches found between the two documents."
- **Don't present the broad research numbers as your exact problem.** Studies showing about half of patients have medication discrepancies after discharge cover all kinds of discrepancies. Summary-vs-prescription mismatches are one slice. Say "one important slice of a big problem," or quote a study specifically on summary-vs-prescription discrepancies if you find one.

Likely judge questions

Question	Answer
Why not just a Python script?	The documents are free text, abbreviations, brand names, sometimes messy scans. Nemotron reads them; Python does the strict comparison; the agent decides what to do when reading fails (retry, wait for the missing file, escalate) and explains each flag.
What if the AI misreads a dose?	Every extracted value must be found inside the exact line it came from. If not, the agent retries once, then sends the case to human review. The AI never gets to "make up" a value.
Won't pharmacists get too many alerts?	Flags are graded. Brand-only differences are low severity and grouped separately. High severity is reserved for different drugs, doses, frequencies, or missing drugs.
Why not use Epic's built-in tools?	Our target hospitals are small, often not on Epic, and many of their documents arrive as PDFs and scans.
What's next?	Add the patient's home medication list as a third input, to catch drugs paused in hospital and never restarted. Then nursing-home intake. Then EHR integration via FHIR.

2. The idea in detail

Scope

ScriptGuard checks **consistency between two documents written by the same discharging team**:

1. Discharge summary (the plan)
2. Discharge prescription (what will be dispensed)

They must agree exactly. Any difference is flagged.

In scope: dose, frequency, route, drug name (including brand vs generic), drugs present in one document only, drugs marked "stop" in the summary but still on the prescription.

Out of scope (roadmap): comparing against the home medication list, drug-drug interactions, kidney-based dosing, allergy checks.

Who uses it

- **Pharmacist:** opens a dashboard of discharges, sees flags ranked by severity, resolves each one.
 - **Discharging doctor:** gets called or messaged when a high flag needs correction.
 - **Hospital:** fewer errors leaving the building, fewer return visits.
-

3. Analysis

Where it fits

- **Small and mid-sized community hospitals** (roughly 25–200 beds) with few pharmacists who can't manually recheck every discharge.
- **Hospitals where summary and prescription are produced separately**, by different people or in different systems. That's where handoff errors live.
- **Hospitals receiving or producing documents as PDFs and scans**, where a language model is genuinely needed to read them.
- **Settings with strict data rules** that prefer nothing leaves the building.

Where it doesn't fit

- **Large hospitals on fully integrated EHRs** (e.g., Epic), where the prescription is generated directly from the same order list as the summary. Mismatches are rarer there, and built-in tools already exist.
- **Hospitals with no IT staff** to keep a local machine running and updated.
- **Anyone expecting it to replace a pharmacist.** It reduces their workload; it doesn't make decisions.

What exists in the market

- **EHR-native medication reconciliation** (Epic, Oracle Health, MEDITECH): built in, but only for hospitals on those systems, and focused on structured data.
- **Medication history services** (DrFirst MedHx, Surescripts): pull pharmacy fill history, mainly for the admission side.
- **Shared-record platforms** (e.g., XchangePoint): share medication histories across facilities; depend on everyone joining the network.

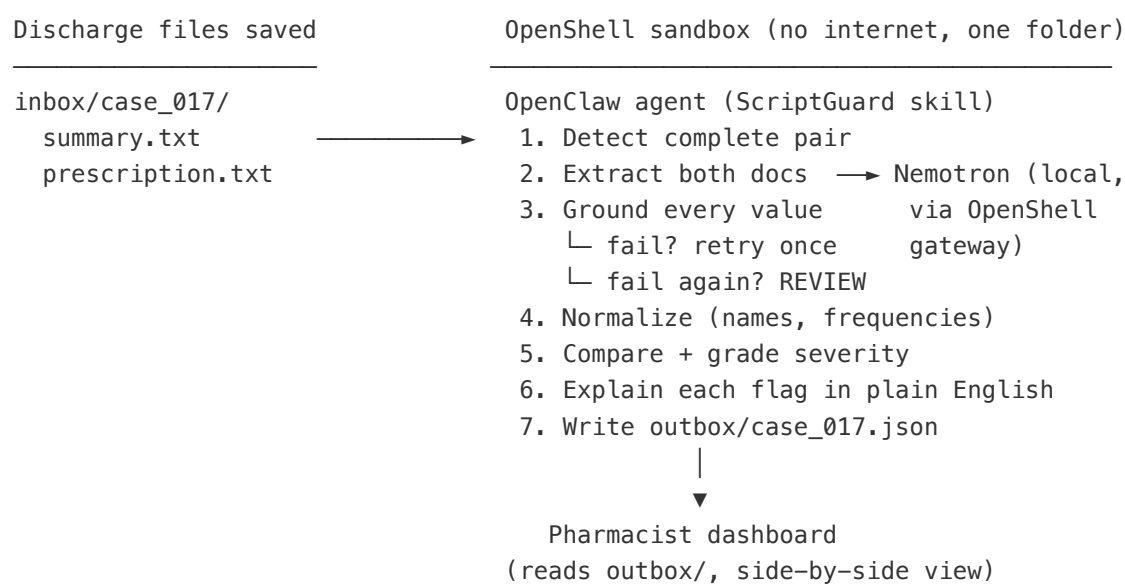
- **AI discharge tools** (e.g., MayaMD's MayaRED): patient-facing discharge plans and education, not pharmacist-facing document checking.
- **Custom AI builds** by consultancies: cloud-based reconciliation agents built to order.

ScriptGuard's angle: a pharmacist-facing, fully local, document-level consistency checker that works on messy text without EHR integration.

Risks and how we handle them

Risk	Mitigation
Model misreads a value	Line-level grounding; retry once; then human review
Model attaches a dose to the wrong drug	All values must come from the same line (span), not just appear somewhere
Alert fatigue	Severity grading; brand-only differences grouped as low
False comfort from "no flags"	Wording: "no mismatches found in N items checked," never "safe"
Brand names not in our lookup	Unknown name → treated as a different drug → high flag. Fails safe
Stack is alpha software	Pin versions; have a fallback trigger and a recorded demo video

4. Technical workflow



Responsibilities, matching your architecture diagram:

- **NemoClaw (setup layer):** installs and launches the OpenShell sandbox, applies the policy, installs OpenClaw, and wires inference to the local Nemotron model. Not part of the runtime loop.
- **OpenShell (sandbox):** enforces no internet, filesystem access limited to the ScriptGuard folder, and routes model calls through its gateway.

- **OpenClaw (agent):** runs the loop, calls tools, makes decisions (retry, wait, escalate), writes results.
- **Nemotron (LLM):** extraction from free text, and plain-English explanations of each flag.
- **Python tools (deterministic):** grounding, normalization, comparison, severity. The model never decides whether two values match.

Where the agent makes real decisions

These are what make it an agent rather than a script. Show at least two in the demo.

1. **Pair incomplete:** summary arrived but prescription hasn't. Agent waits and rechecks instead of failing.
2. **Grounding failed:** agent re-runs extraction with the grounding error fed back ("dose '500' not found in line 3"), once. Then escalates.
3. **Explaining flags:** agent writes a one-sentence explanation per flag for the pharmacist.
4. **Escalation:** unreadable document → marks case REVIEW with a reason instead of guessing.

5. Tech stack

Layer	Choice	Notes
Hardware	Dell Pro Max GB10	Everything runs here, offline
Setup / lifecycle	NemoClaw	Onboards sandbox, policy, agent, inference profile
Sandbox	OpenShell	Policy: no egress except inference gateway; one folder
Agent runtime	OpenClaw	ScriptGuard skill + tool calls
LLM	Nemotron (local)	Use whichever Nemotron model you pre-downloaded and served locally. Smaller = faster demo; temperature 0
Deterministic tools	Python 3 (stdlib only)	<code>extract.py</code> , <code>ground.py</code> , <code>normalize.py</code> , <code>compare.py</code>
Data	Plain JSON files	No database needed
Dashboard	Single HTML page or small Flask app	Reads <code>outbox/*.json</code> , polls every 2 s

Don't add anything else in 4 hours.

6. Data contracts

Folder layout

`scriptguard/`

```
inbox/case_017/summary.txt
inbox/case_017/prescription.txt
outbox/case_017.json
tools/extract.py ground.py normalize.py compare.py run_case.py
data/brands.json data/frequencies.json
dashboard/
```

Extraction output (per document)

```
{
  "doc": "summary",
  "meds": [
    {
      "line": 3,
      "span": "Continue metformin 500 mg BID",
      "name_raw": "metformin",
      "dose_raw": "500",
      "unit_raw": "mg",
      "freq_raw": "BID",
      "route_raw": null,
      "status": "continue"
    }
  ]
}
```

status is one of continue, start, stop, change. Raw fields are copied exactly as written. Normalization happens later, in Python.

Result (outbox/case_017.json)

```
{
  "case": "case_017",
  "status": "FLAGGED",
  "checked": 3,
  "flags": [
    {
      "severity": "HIGH", "drug": "metformin", "type": "dose_mismatch",
      "summary": "metformin 500 mg BID (line 1)",
      "prescription": "Metformin 1000 mg BID (line 1)",
      "explanation": "The prescription doubles the dose stated in the summary."
    }
  ]
}
```

status is NO_MISMATCHES, FLAGGED, or REVIEW (couldn't read reliably).

7. Core logic

Extraction prompt (Nemotron, temperature 0)

You extract medications from a clinical document.
Return ONLY JSON matching this schema, no other text: {schema}
Rules:

- One entry per medication line.
- "line" is the 1-based line number in the document.

- "span" is copied character-for-character from that line.
 - name_raw, dose_raw, unit_raw, freq_raw must be copied exactly as written inside the span. Do NOT expand abbreviations or convert brand names.
 - If a field is not present, use null.
- Document (numbered lines):
{numbered_text}

Grounding

```
def ground(med, lines):
    line = lines[med["line"] - 1].lower()
    span = med["span"].lower()
    if span not in line:
        return f"span not found on line {med['line']}"
    for f in ("name_raw", "dose_raw", "unit_raw", "freq_raw", "route_raw"):
        v = med.get(f)
        if v and v.lower() not in span:
            return f"{f} '{v}' not inside its span"
    return None # grounded
```

Normalization

```
BRANDS = {"dolo 650": ("paracetamol", "650"), "dolo": ("paracetamol", None),
          "crocin": ("paracetamol", None), "calpol": ("paracetamol", None),
          "tylenol": ("paracetamol", None), "acetaminophen": ("paracetamol", None),
          "glycomet": ("metformin", None), "lasix": ("furosemide", None),
          "eltroxin": ("levothyroxine", None), "thyronorm": ("levothyroxine", None)}
FREQ = {"od": 1, "qd": 1, "once daily": 1, "bd": 2, "bid": 2, "twice daily": 2,
        "tds": 3, "tid": 3, "three times daily": 3, "qid": 4, "prn": "prn", "hs": "night"}

def normalize(med):
    name = med["name_raw"].lower().strip()
    ingredient, implied_dose = BRANDS.get(name, (name, None))
    dose = med["dose_raw"] or implied_dose
    return {"name": name, "ingredient": ingredient,
            "is_brand": name in BRANDS and name != ingredient,
            "dose": dose, "unit": (med["unit_raw"] or "mg").lower(),
            "freq": FREQ.get((med["freq_raw"] or "").lower(), med["freq_raw"]),
            "status": med["status"]}
```

Comparison and severity

```
def compare(summary, rx):
    flags = []
    s_active = {m["ingredient"]: m for m in summary if m["status"] != "stop"}
    s_stopped = {m["ingredient"] for m in summary if m["status"] == "stop"}
    r = {m["ingredient"]: m for m in rx}
    for ing, s in s_active.items():
        p = r.get(ing)
        if not p:
            flags.append(("HIGH", ing, "missing_from_prescription")); continue
        if (s["dose"], s["unit"]) != (p["dose"], p["unit"]):
            flags.append(("HIGH", ing, "dose_mismatch"))
        if s["freq"] != p["freq"]:
```

```

        flags.append(("HIGH", ing, "frequency_mismatch"))
    if s["name"] != p["name"]:
        flags.append(("LOW", ing, "name_differs_confirm_brand_intent"))
    for ing in r:
        if ing in s_stopped:
            flags.append(("HIGH", ing, "stopped_in_summary_but_prescribed"))
        elif ing not in s_active:
            flags.append(("HIGH", ing, "not_in_summary"))
    return flags

```

An unknown brand name maps to itself, so it never matches anything and gets a HIGH flag. That's the safe direction.

8. Demo cases

Write these as text files before you start building. They are your tests.

Case	What's in it	Expected result
Mary (hero case)	Summary: metformin 500 BID, sertraline 50 OD, Dolo 650 TDS. Rx: metformin 1000 BID, paracetamol 650 TDS	HIGH dose mismatch (metformin), HIGH missing (sertraline), LOW brand (Dolo vs paracetamol)
Clean	Both documents identical	NO_MISMATCHES
Stopped but prescribed	Summary: "stop amlodipine". Rx includes amlodipine 5 mg OD	HIGH stopped_in_summary_but_prescribed
Brand-critical	Summary: Eltroxin 50 mcg OD. Rx: levothyroxine 50 mcg OD	LOW brand flag, explanation mentions brand switching for thyroid medicine
Messy / unreadable	Summary missing a dose, garbled line	Agent retries, then REVIEW with reason

9. Four-hour build plan

Assumes 3 people: **A** (stack + agent), **B** (Python logic), **C** (dashboard + pitch). Adjust if your team is different.

Hour 0:00–0:30 — Stack check (everyone, A leads)

- NemoClaw sandbox launches; OpenClaw agent responds to a test message.
- Nemotron answers through the OpenShell gateway from inside the sandbox. Note the endpoint URL and model name.
- Sandbox policy allows read/write to the `scriptguard/` folder only. Confirm the agent can list `inbox/`.
- Decide how files move between host and sandbox: shared folder. the CLI's file

upload/download, or run the dashboard inside the sandbox with a forwarded port. Pick whichever works first.

-  C writes all five demo cases into `inbox/`.

Checkpoint: if Nemotron can't be reached from the sandbox by 0:30, everyone helps A until it works. Nothing else matters without it.

Hour 0:30–1:45 — Build in parallel

- **A:** `extract.py`: numbered lines in, JSON out, using the prompt above. Parse failures retry once. Test on the Mary summary.
- **B:** `ground.py`, `normalize.py`, `compare.py`, `run_case.py` (chains them, writes `outbox/`). Test with hand-written extraction JSON so you don't depend on A.
- **C:** dashboard. List of cases with status badges; click a case to see summary and prescription side by side with flagged lines highlighted and the explanation. Hardcode a sample result file to build against.

Checkpoint at 1:45: `run_case.py case_mary` produces the expected three flags end-to-end.

Hour 1:45–2:45 — Make it an agent

- **A + B:** OpenClaw ScriptGuard skill. Instructions to the agent:
 1. Check `inbox/` for case folders with both files. If only one, note it and check again later.
 2. Run extraction; run grounding.
 3. If grounding fails, re-run extraction with the error message included; if it fails twice, write REVIEW with the reason.
 4. Run compare; for each flag, write a one-sentence explanation.
 5. Write the result to `outbox/` and move the case to `inbox/done/`.
- **Trigger:** use OpenClaw's scheduled/heartbeat check if it works in your version. Fallback: a host-side loop that messages the agent "process inbox" every 10 s. Fallback 2: trigger by message during the demo and say "in production this runs on a schedule."
- **C:** connect dashboard to real `outbox/` files.

Checkpoint at 2:45: drop the Mary folder into `inbox/`, dashboard shows flags without anyone touching anything.

Hour 2:45–3:30 — Harden and test

-  Run all five demo cases. Fix wrong results.
-  Time the Mary case end-to-end. If it's slow, cut the explanation step to one call per case, not per flag.
-  Show the sandbox policy in action: have the agent try to reach the internet and show it being blocked. Judges like this.
-  Record a backup video of the full demo.

Hour 3:30–4:00 — Pitch

- C finalizes slides: problem, Mary story, live demo, why every mismatch is flagged, why local, roadmap.